Project Proposal - GSOC 2025

Physics Informed Neural Network Diffusion Equation (PINNDE)

Applicant : Sijil Jose

Mentor Organization: ML4SCI

Corresponding Project : GENIE

Submitted as part of the Application for GSOC 2025

April 8, 2025

Contents

2 Project Description

2.1 Abstract

Sampling from complex, high-dimensional distributions remains a core challenge in both machine learning and computational physics, with key applications spanning generative modeling and rapid simulation. Traditional methods in physics, such as MCMC and nested sampling, typically depend on evaluating likelihood functions or model evidence—tasks that are not always feasible or computationally tractable.

In contrast, generative probabilistic models in machine learning aim to learn a target distribution using a finite set of samples, enabling both the generation of new samples and the evaluation of probability densities. Diffusion probabilistic models, in particular, have gained significant attention in recent years. They offer advantages over other deep learning approaches—like GANs, which can be difficult to train, and normalizing flows, which impose strict invertibility constraints on network architectures.

Sampling from diffusion probabilistic models is commonly approached by solving the corresponding reverse-time diffusion ordinary differential equation (ODE) using finite-difference solvers. In this project, we propose an alternative approach: leveraging physics-informed neural networks (PINNs) to solve the reverse-time diffusion ODE across a range of parameters. Although PINNs require a substantial upfront training cost, this cost is amortized during inference, enabling extremely fast sampling from the learned distribution.

Our initial focus will be on developing proof-of-concept models for various 3D probability density functions (PDFs) to evaluate different strategies. Ultimately, we aim to apply this framework to the fast calorimeter simulation challenge, potentially providing a powerful tool for addressing computational challenges in particle physics. This project sits at the intersection of cutting-edge machine learning techniques and experimental particle physics, with significant potential to advance both fields.

2.2 Methodologies and Literature Review

In this section I will very briefly comment on the Diffusion probabilistic models and different variants of PINN that can be used for solving the Diffusion ODE.

2.2.1 Diffusion Probabilistic Models

DPMs work by building a process $x_{t\in[0,T]}$ which takes samples x_0 from a probability distribution $q_0(x_0)$ of interests and maps to simple noise distribution like multi variable gaussian starting from t=0 to t=T . This process is described by a stochastic differential equation of the following form.

$$dx_t = f(t)x_t dt + g(t)d\omega_t, \qquad x_0 \sim q_0(x_0) \tag{1}$$

Under some regularity conditions , this forward process has an equivalent reverse process from t = T to t = 0. These are usually called as score based diffusion models [10]]. The probability flow ODE is a deterministic counterpart to the reverse SDE. While the SDE includes a noise term (the Wiener process), the ODE removes this randomness, yet is designed to have the same marginal probability distributions as the SDE.

$$\frac{dx_t}{dt} = h_\theta(x_t, t) = f(t)x_t + \frac{g^2(t)}{2\sigma_t}\epsilon_\theta(x_t, t), \qquad x_t \sim \mathcal{N}(0, \sigma^2 I) \tag{2}$$

Samples can be drawn by solving the ODE from T to 0. Since the probability flow ODE is deterministic, integrating it from t=T back to t=0 gives a unique trajectory for each initial condition. This deterministic mapping is valuable as in contrast with SDEs, ODEs can be solved with larger step sizes as they have no randomness [5]. Our goal in this project would be solve the ODEs of the above form using physics informed neural network(PINN) which can be used to approximate the solutions for a range of values, so that after training the PINN, we can generate fast samples from our distribution of interest.

2.2.2 Physics informed Neural Networks

Physics informed neural networks offers a unique approach to incorporate the physical laws of the system of interest in the form of differential equations into training a neural network. The core idea of PINNs is to leverage the universal function approximation capabilities of neural networks while constraining them with known physics. so, while training a PINN, the solution of the differential equation is approximated with a neural network (very similar in philosophy with spectral method in numerical PDE solvers), and the neural network is trained with a physical loss terms which calculates how well the network satisfies the diffeential equation of interest. This approach allows for the solution of both forward problems (predicting system behavior given initial conditions) and inverse problems (deducing system parameters from observed data). By incorporating physical laws into the learning process, PINNs can effectively handle scenarios with limited or noisy data, as the embedded physics provides additional guidance to the network. They provide a mesh-free alternative to traditional numerical methods, can seamlessly incorporate boundary and initial conditions, and are capable of solving high-dimensional problems that are often intractable with classical techniques. Moreover, once trained, PINNs offer rapid evaluations, making them suitable for real-time simulations and control applications. We plan to take advantage of these properties of PINN into solving probability flow ODE of diffusion models. Training a PINN is often a multitask optimisation problem where we have two loss terms, one corresponding to the differential equation and the other corresponding to the initial conditions and data points that are available. we can try to rewrite the PINN in such a way that they are hard constrained to satisfy the initial or boundary conditions(**as shown in the Jupyter notebook corresponding to the test**). In this project I am interested in implementing the following PINN variants to tackle the probability flow ODE.

- **Vanilla PINN :** First I will be implementing the vanilla PINN (No additional training data - only Physical loss term and initial/boundary conditions) with different architectures to develop a benchmark and understand the regions in the solution space which PINN finds hard to constrain.[8]

- **Hard constraints for Initial/Boundary conditions :**.This ensures that solution from PINN always satisfies the initial/boundary conditions, thus transforming the multiobjective training to single objective problem. [example shown in the test notebook]

- **Adding additional training data :** If , PINN isn't able to represent the solution well, we can try guiding the PINN by adding the numerical solution of the ODE on a grid of values (or regions where PINN performs poorly) as training data.

- **Augmented Lagrangian Method (ALM) :** In the cases where we have two loss terms, I would be using the Augmented Lagrangian methods suited for dealing with constrained optimization problems (AL-PINN). Generally speaking, ALM combines the merits of the penalty and Lagrange multiplier methods while avoiding the ill conditioning and convergence issues associated with these methods. This strategy has been shown to work much better than vanilla PINN in various benchmarks.[9] [1]

- **Coupled automatic-numerical PINN :** This method tries to stabilize the PINN solution by incorporating coupling between neighboring collocation points and their derivatives. They have been shown to improve the PINN accuracy, and require fewer collocation points while training. This unifies the advantages of Automatic differentiation and Numerical Differentiation, providing more robust and efficient training than AD-based PINNs. [3]

- **Operator learning frameworks :** Neural operator methods have emerged as a powerful framework for solving ordinary differential equations (ODEs) by learning mappings between function spaces. They work on a completely different principle compared to PINN. Neural operator methods aim to learn an operator ie. mapping between function spaces. Instead of learning the solution for just one instance of an

ODE, neural operators are designed to generalize across entire families of problems. They learn how changes in input functions (for example, initial conditions, forcing terms, or other parameters) translate into changes in the solution function. This formalism should be perfect match for this problem where we try to learn the ODE solution for a range of parameters and initial conditions. [6] [2]

2.2.3 Optimization Strategies :

Here I will Briefly mention the different Optimization strategies that I plan to incorporate while working on this project. As mentioned in the previous section, there are cases where training PINN can be treated as single objective optimization tasks (cases where initial/boundary conditions are hard constraints) or Multiobjective optimization tasks (cases where we have both loss terms from training data and physical loss term). These two cases should to be treated differently because the naive approach of combining different loss terms with different weights will eventually cause biases as the optimizer could focus on just one of the loss terms if their gradients points at different direction. So, in the multi objective optimisation tasks, we need to use strategies that are **Pareto optimal** .

- **Single Objective Optimization :** In the cases where in the initial/boundary conditions are hard constrained , we can use the standard techniques for training the PINN. The most common strategy followed in the community and has been shown to work quite well is start with the adam optimizer and then shift to L-BFGS in the end. [4]
- **PC grad (Gradient surgery for multi task learning) :** The PCGrad algorithm—short for "Projected Conflicting Gradients"—is a technique designed to mitigate the interference that arises when training multi-task neural networks. In multi-task learning, different tasks share a common set of network parameters but naturally produce task-specific gradients that can conflict with one another, ultimately hampering the overall performance. PCGrad addresses this issue by modifying the individual gradients by de-projecting them , so that they are more aligned with each other before being aggregated for the update step.[11]
- **Nash bargaining equilibrium solution :** This strategy reinterprets multi-task learning as a game in which each task acts as an independent "player" negotiating over a shared resource—the common model parameters. In this framework, each task seeks to optimize its own objective, but since these objectives often conflict, simply summing losses can lead to suboptimal compromises. Viewing the multi-task problem as a bargaining game, the paper leverages tools from game theory and multi-objective optimization to formulate the learning process as a negotiation

among tasks. This has been shown to converge to Pareto optimal solution for a multi task optimization problem. [7]

2.3 Benefits to the Community:

This project aims to integrate two state of the art methods namely, the diffusion probabilistic model and Physics informed neural networks to build a fast sampler for high dimensional probability distribution. Sampling from a complex high dimension probability distribution is often a bottleneck in many scientific and industrial applications like physics simulators, Bayesian inference and generative modeling. Successful completion of this project will produce a proof of concept upon which others can improve upon. We plan to implement this strategy for the Fast calorimeter simulation challenge (2022), an initiative aimed at developing fast, high-fidelity simulation techniques for calorimeter detectors—key components in high-energy physics experiments such as those conducted at the Large Hadron Collider (LHC). By accelerating the simulation process, this project has the potential to significantly impact experimental physics. Fast and accurate simulations are a key resource for designing detectors, optimizing experiments, and analyzing data, thereby aiding in the discovery of new physics phenomena.

3 Project Proposal

3.1 Overview

This proposal outlines a two phased approach to implement the above described PINNDE (Physics informed neural network - Diffusion Equation) strategy. divided in 8 weeks and 4 weeks respectively.

- **Phase 1 :** Phase 1 will focus on preparing a proof of concept for integrating Physics informed neural network into Diffusion probabilistic models (PINNDE). This will be focused on implementing the methodologies mentioned in project description section (From simplest to advanced) to systematically study the effects of different implementation. In this Phase , I will be working on mapping a 3D normal distribution to a 3D non-gaussian distributions using the PINNDE method. Basically I will be mapping the inputs to the PINN (t, x, y, z) — that is, the reverse time $t \in [1, 0]$ and a point sampled from the 3D normal to values u(t, x, y, z) which would be samples from the target distribution. The following target distributions will be considered to test different capabilities of the method.
  - **Mixture of Gaussian :** This test the viability of the method on multimodal distributions.

- **3D spiral/helical Distribution :** This can be generated by parametric functions for each coordinates like $x(t) = cos(t), y(t) = sin(t), z(t) = \frac{t}{2\pi}$. This serves as agood test for PINNDE's capabilities for capturing both linear and cyclical patterns.
- **Toroidal :** This introduces a nontrivial topology (e.g., a hole in the middle) which provides a more challenging, yet still controlled, structure for a diffusion model to learn.
- **Concentric spherical shells :** This tests the model's ability to learn distributions with intrinsic spherical symmetry and discontinuities in density (sparse regions between shells), which are common in physical simulations such as in astrophysics.

Since I already have some experience with PINNs and the above mentioned extensions, I should be able to get the different models running soon.

- **Phase 2 :** The goals of this phase will be highly dependent on the results from the tests on proof of concepts from Phase 1.
  - **If methods implemented in phase one manages to pass the tests in phase 1 :** In this case I plan to start working on the Fast calorimeter simulation challenge 2022. The fast calorimeter simulation challenge data set is designed to be a standardized benchmark for developing and comparing fast surrogate models of calorimeter simulations, which are critical components in high-energy physics experiments. The dataset is derived from detailed simulations (typically using tools like Geant4) and is structured to capture the complex spatial patterns and energy deposition profiles resulting from particle interactions in a calorimeter. The challenge offers three datasets, ranging in difficulty from "easy" to "medium" to "hard". The difficulty is set by the dimensionality of the calorimeter showers (the number of layers and the number of voxels in each layer). I will be focusing on stress testing the methodologies in phase 1 on the easy dataset, and move to more complex data depending on the results and available time after this.
  - **If the phase 1 doesn't provide robust results :** In this case I will be channeling my efforts into understanding why the PINNDE method fails, and decide on the next steps based on discussion with the mentors.

Computational resources : As I am enrolled at SISSA as a PhD student, I have access to HPC clusters with GPU nodes at SISSA and I plan to use the same for training the models during both phases of the project.

3.2 Plan of Action

3.2.1 Phase 1 tasks :

- Generate mock datasets to train , test and evaluate the capabilities of PINNDE strategy. (described in project overview)
- Implement different metrics to quantify how well the model represents the true probability distributions. [Maximum Mean Discrepancy , Wasserstein Distance]
- Implement different metrics to quantify how well the PINN represents the solution of the probability flow ODE [ODE residue on grids, Mean square error from initial/boundary or training data , similar tests on test datasets.]
- Implement visualization to view how well the PINNDE model represents the true distribution
- Implement visualization to view how well the PINN model represents the solution of probability flow ODE.
- Implement and train successively complex models mentioned in the methodologies section (starting from vanilla PINN to neural operator based methods) on all the test distributions perform the implemented tests and visualizations.

3.2.2 Phase 1 Deliverables :

- Diverse datasets to test all the Implemented models.
- Detailed quantitative comparison of implemented extensions of PINN models 's ability to represent probability flow ODEs.
- Comparison of sampling speed , accuracy and computational costs and training time, for each model
- a detailed report with insights into pro and cons of each implemented PINN variant.

3.2.3 Phase 2 tasks :

- Modify the models trained in phase 1 to extend and test them on Fast calorimeter simulation challenge dataset 1
- Train and test the different implantation of PINNDE method on dataset 1.
- Run all the metric considered in the data challenge 2022.
- Comparison with existing rank lists from the data challenge 2022.

3.2.4 Phase 2 Deliverables :

- Results/Metrics of all the variants of PINNDE variants trained on dataset 1 required for the .
- ranking of PINNDE with respect to rank list from the 2022 data challenge.
- a detailed report with insights into pro and cons of each implemented PINN variant for the Fast calorimeter simulation data challenge 2022.

3.3 Timeline

- **week 1 :** Generating training and test data for all the 4 test 3D distribution; Implementation of tests/metrics and plotting routines; Implementation and Training of vanilla PINN and PINN with hard constraints on initial/boundary conditions on all 4 distributions.
- **week 2 and week 3** : Adding additional training data obtained from numerically solving the ODE on a grid of values using traditional numerical solvers; Implementing and training can-PINN and augmented Lagrangian method for training PINN; Hyper parameter tuning of models implemented in week 1.
- **week 4 , week 5 and week 6 :** Implement neural Operator based methods for probability flow ODEs. Incorporate PC grad / Nash bargaining solution for multi objective tasks with training data; Hyperparameter tuning of all the promising models.
- **week 7 and week 8 :** Finish Implementation of any of the models if not completed by week 6 ; Organize and evaluate the results from Phase 1; Literature review for further improvements in phase 2.
- **week 9 and week 10 :** Depending on the results from Phase 1, implement the promising models for Fast calorimeter dataset 1.
- **week 11 and week 12 :** Hyper parameter tuning, Analyzing the results and finalizing the project reports.

4 Commitments :

Since I am an active PhD student I have commitments from my thesis projects throughout this 12 week period. I will be able to allocate 1-1.5 hrs on the working days and 4-5 hours on the weekend on average. This time allocation allows me to easily satisfy the 175 hour requirement in the project guidelines.

5 Additional Information on the timeline

The timeline mentioned above is subject to change and is only an approximate outline of my project work. I will stick to or exceed this schedule and create a more detailed schedule during the pre-GSoC and community bonding phase. Time will be divided (according to workload) each week among planning, learning, coding, documenting and testing features. All documentation will go hand in hand with the development.

6 Post GSOC Plans

I am committed to the long term goal of ML4SCI of developing machine learning based solutions for tackling computational problems in science. I am also inspired by the open source commitment of the organization. I plan to extend this methodology to learning conditional probability densities and thus enabling to use them in simulation based inferences for parameter estimation of cosmological parameters. I would like to also contribute other open source initiatives of ML4SCI and contribute as a mentor in the upcoming years.